HW7 Solution

Problem 1

Let $f(x) = 3x_1^2 + 4x_2^2 - x_1x_2 - 5x_1 - 8x_2$.

Then the gradient of $f(x)$ is $\nabla f(x) = \begin{bmatrix} 6x_1 - x_2 - 5 \\ 8x_2 - x_1 - 8 \end{bmatrix}$.

Suppose we start at $x_0 = (0,0)^T$, it is easy to find the gradient of this point $\nabla f(x_0) = \begin{bmatrix} -5 \\ -8 \end{bmatrix}$.

Using the gradient update rule, we have $x = x_0 - \alpha \nabla f\left(x_0\right)$.

At initial points $(0,0)^T$, we have $x = \alpha \begin{bmatrix} 5 \\ 8 \end{bmatrix}$.

Using exact line search, we need to minimize $f(5\alpha, 8\alpha)$, which is $291\alpha^2 - 89\alpha$.

So, $\alpha = 89/582$.

Bring back to update equation, we have

$x = 89/582 \times \begin{bmatrix} 5 \\ 8 \end{bmatrix} = \begin{bmatrix} 445/582 \\ 356/291 \end{bmatrix} = \begin{bmatrix} 0.765 \\ 1.223 \end{bmatrix}$.

problem 2

Notice that the golden section method does not require knowledge of f'. Hence, applying the golden section method can save some preparatory calculations. We can run and compare the bisection and golden section method for Problem 2:

```
options
options.maxit = 100;
options.tol = 1e-5;
options.display = true;

% function
f = @(x) -(log(x)-2*(x-1)/(x+1))/(x-1)^2;
g = @(x) 2*(log(x)-2*(x-1)/(x+1))/(x-1)^3-(1/x-4/(x+1)^2)/(x-1)^2;

xl = 1.5;
xr = 4.5;

[xb,gx] = bisection(g,xl,xr,options);
[xa] = ausection(f,xl,xr,options);
```

The golden section method returns the solution $x_{\mathrm{g}}^* = 2.18870459$ after 27 iterations. The bisection method requires 20 iterations and it returns $x_{\mathrm{b}}^* = 2.18870258$.

Problem 3

(a)

$$\nabla f = \left(12 + 4x_1 - 10x_2 + 4x_2^2 - 2x_2^3, -2\left(x_1\left(3x_2^2 - 4x_2 + 5\right) - 3x_2^5 + 5x_2^4 - 12x_2^3 + 24x_2^2 - 25x_2 + 15\right)\right)$$

You can use the condition $\nabla f = 0$ to verify all the stationary point.

(b)

$$\nabla^2 f(x) = \begin{bmatrix} 4 & -6{x_2}^2 + 8x_2 - 10 \\ -10 + 8x_2 - 6x_2^2 & -12x_1x_2 + 8x_1 + 30x_2^4 - 40x_2^3 + 72x_2^2 - 96x_2 + 50 \end{bmatrix}$$

For ${x_1}^*$, $\nabla^2 f(x) = \begin{bmatrix} 4 & -10 \\ -10 & 74 \end{bmatrix}$ $\nabla^2 f(x)$ is PD, so minimizer, $f\left({x_1}^*\right) = 72$

For $x_2^*, \nabla^* f(x) = \begin{bmatrix} 4 & -8 \\ -8 & 20 \end{bmatrix}$, $\nabla^2 f(x)$ is PD, so minimizer, $f\left(x_2^*\right) = 0$

For $x_3^*, \nabla^2 f(x) = \begin{bmatrix} 4 & -24 \\ -24 & 148 \end{bmatrix}, \nabla^2 f(x)$ is PD, so minimizer, $f\left(x_3^*\right) = 0$

For $x_4^*, \nabla^2 f(x) = \begin{bmatrix} 4 & -12 + \frac{8\sqrt{3}}{3} \\ -12 + \frac{8\sqrt{3}}{3} & 12.2872 \end{bmatrix}, \nabla^2 f(x)$ is indefinite matrix, so saddle point, $f\left(x_4^*\right) = 0.0741$

For $x_5^*, \nabla f(x) = \begin{bmatrix} 4 & -12 - \frac{8\sqrt{3}}{3} \\ -12 - \frac{8\sqrt{3}}{3} & 67.7128 \end{bmatrix}, \nabla^2 f(x)$ is indefinite matrix, so saddle point, $f\left(x_5^*\right) = 0.0741$

In conclusion, ${x_1}^*$ is local minimizer, ${x_2}^* x_3^*$ are global minimizers, x_4^*, x_5^* are saddle points.

(c)

Results:

```
>> gradient_descent
Which strategy to use?
 [1]: large constant stepsize
 [2]: small constant stepsize
 [3]: exact line search
 [4]: armijo backtracking
3
The algorithm ends after 117 iterations
 x*=[-1.000010,0.999996]
 f(x*)=0.000000
 gradient norm=9.663330e-06
>> gradient_descent
Which strategy to use?
 [1]: large constant stepsize
 [2]: small constant stepsize
 [3]: exact line search
 [4]: armijo backtracking
4
The algorithm ends after 162 iterations
 x*=[-0.999992,1.000004]
 f(x*)=0.000000
 gradient norm=9.419326e-06
>>
```

Codes:

```
%common parameters
x = [0; 0];
epsilon = 10^-5;        % stopping  criteira
maxiter = 1000;          % max iteration
iter = 0;
%parameters for exact line search
% Here, we use golden Section search to find the minimizer
e = 10^(-6);            % Accuracy of solden section search.
search_range=1     ;    % range for searching the step length.

%parameters for Armijo backtraking
alpha = 0.1;
beta = 0.5;
t = 1;  %initial step length
%function
f = @(x) (3 + x(1) + ((1 - x(2)) * x(2) - 2) * x(2))^2 + (3 + x(1) + (x(2) - 3) * x(2))^2;

prompt = 'Which strategy to use?\n [1]: large constant stepsize\n [2]: small constant stepsize\n [3]: exact lin
s = input(prompt);

%plot the contour of cost function:
x1 =  -2.5:0.01:0.5;
x2 =   0:0.01:3;
n=length(x1);
z=zeros(n,n);
for i=1:n
    for j=1:n
        z(n+1-i,n+1-j)=f([x1(i),x2(j)]);
    end
end
figure(1)
contour(x1,x2,z,20);
hold on
%gradient descent algorithm:
while norm(gradient(x)) > epsilon && iter<=maxiter
    d =  -gradient(x);
    if s==1
        alpha_k = 0.1;       %large constant step size
    elseif s==2
        alpha_k = 0.001;     %small constant step size
    elseif s==3
        alpha_k = exact_line(f,x,d,e,search_range); %exact_line_search
    else
        alpha_k = armijo_backtracking(f,x,d,alpha,beta,t);%armijo backtracking
    end
    xtemp = x + alpha_k * d;
    % Make some plots
    plot(x(1), x(2), '*r');
    hold on;
    plot([x(1), xtemp(1)], [x(2), xtemp(2)], '-g');
```

```
    hold on;
    % Output the solution in each step
    iter = iter + 1;
    x = xtemp;
end

fprintf('The algorithm ends after %d iterations\n x*=[%f,%f]\n f(x*)=%f\n gradient norm=%e\n',iter,x,f(x),norm(




function y = gradient(x)
    x1 = x(1);
    x2 = x(2);

    y1 = 12 + 4 * x1 - 10 * x2 + 4 * x2 ^ 2 - 2 * x2 ^ 3;
    y2 = -2 * (x1 * ( 3 * x2^2 - 4 * x2 + 5) - 3 * x2 ^ 5 + 5 * x2 ^ 4 - 12 * x2 ^ 3 + 24 * x2 ^ 2 - 25 * x2 +


    y = [y1; y2];
end


function alpha_k = exact_line(f,x,d,e,search_range)
    phi = (3-sqrt(5))/2;
    alpha_l = 0;
    alpha_r = search_range;
    alpha_l0 = phi * alpha_r + (1-phi) * alpha_l;      % find a left point in the section using the golden rati
    alpha_r0 = (1-phi) * alpha_r + phi * alpha_l;
    f_1 = f(x+alpha_l0*d);
    f_2 = f(x+alpha_r0*d);

    while alpha_r-alpha_l > e
        if f_1 < f_2        % If the function value is greater toward the left,
            alpha_r  = alpha_r0;
            %Update alphar1 and alphar2
            alpha_r0 = alpha_l0;
            alpha_l0 = phi * alpha_r + (1-phi) * alpha_l;
            %Update function values
            f_2 = f_1;
            f_1 = f(x+alpha_l0*d);
        else
            alpha_l = alpha_l0;
            %update alphar1 and alphar2
            alpha_l0 = alpha_r0;
            alpha_r0 = (1-phi) * alpha_r + phi * alpha_l;
            f_1= f_2;
            f_2 = f(x+alpha_r0*d);
        end
    end
```

```
    alpha_k=(alpha_l+alpha_r)/2;
end
```

```
function alpha_k = armijo_backtracking(f, x, d, alpha, beta, t)
    alpha_k = t; % Initial step size
    while f(x + alpha_k * d) >= f(x) + alpha * alpha_k * gradient(x)' * d
        alpha_k = alpha_k * beta;
        if alpha_k < 1e-6 % prevent alpha_k from becoming too small
            break;
        end
    end
end
```

Problem 4

(a)

(i) True. When $\alpha = 1$, the armijo condition holds.

(ii) True. $\text{The armijo condition is satisfied for all } \alpha \in [0, 1]$.

(iii) False. The global minimizer is not in interval $[0.5, 1]$.

(b)

(i) True. It is the largest stepsize in the set that satisfis armijo condition.

(ii) False. Cannot ensure that point is the global minimizer.

(iii) True. It is the largest stepsize in the set that satisfies armijo condition.